

Statistical Methods Quiz #4 (50 minutes)

'26. 6. 12.(Friday)

Cadet number(교번):

Name:

Score:

<Instructions>

1. It is open-book, closed-web test.
2. You are not allowed to discuss on the problems during the exam.
3. Import `Movie.csv` without converting the string variables to factors. Also, import `Auto.csv`.
4. Run the provided code block (visible below) to prepare data and train a baseline model then answer the questions. Do not change the given R code.

We try to predict the binary variable with two levels *positive*(*class* = 1) and *negative*(*class* = 0) in *Movie.csv* using Korean reviews with a neural network model.

<Baseline Code> - Run This First (Do not change the R code)

```
library(keras3)
set.seed(41)
review = Movie
colnames(review) <- c("score", "comment")
review$score <- as.integer(review$score)
review <- na.omit(review) # Remove NAs
review$score = ifelse(review$score <= 8, 0, 1) # Recode to 0-1 data
# -----
# Train / valid / test split
# -----
n <- nrow(review)
idx <- sample.int(nrow(review), n) # sample n rows from the WHOLE dataset (not just 1:n)
review <- review[idx, ]
n_test <- max(1, round(0.20 * n))
test_set <- review[seq_len(n_test), ]
train_set <- review[-seq_len(n_test), ]
x_train <- train_set$comment
x_test <- test_set$comment
y_train <- train_set$score
y_test <- test_set$score
# -----
# Text vectorization (minimal preprocessing)
# -----
# - Keeps text as-is (no lowercasing/stripping) because it's Korean.
max_words <- 25000L
maxlen <- 50L # you can raise to reduce truncation
vec <- layer_text_vectorization(
  max_tokens = max_words,
  output_mode = "int",
  output_sequence_length = maxlen,
  standardize = NULL # keep Korean punctuation/spacing as-is
)# Build the vocabulary from training text only
adapt(vec, train_set$comment)
# Save the whole vocabulary (not necessary)
vocab <- keras3::get_vocabulary(vec); head(vocab, 30)
# -----
# Model: TextVectorization -> Embedding -> CNN
# -----
embed_dim <- 64L
model <- keras3::keras_model_sequential(
  layers = list(
    vec,
    keras3::layer_embedding(input_dim = max_words, output_dim = embed_dim, mask_zero = FALSE),
    keras3::layer_spatial_dropout_1d(rate = 0.2),
    keras3::layer_conv_1d(filters = 128, kernel_size = 5, activation = "relu", padding = "same"),
    keras3::layer_conv_1d(filters = 128, kernel_size = 5, activation = "relu", padding = "same"),
    keras3::layer_global_max_pooling_1d(),
    keras3::layer_dropout(rate = 0.4),
    keras3::layer_dense(units = 64, activation = "relu"),
    keras3::layer_dropout(rate = 0.3),
    keras3::layer_dense(units = 1, activation = "sigmoid")
  )
)# -----
model |> compile(
  optimizer = "adam",
  loss = "binary_crossentropy",
  metrics = "accuracy"
)# -----
# Train
# -----
history <- model |>
fit(
  x = x_train, y = y_train,
  epochs = 5,
  batch_size = 64,
  validation_split = 0.2,
  verbose = 2
)
```

1. Print a readable model summary. Based on the baseline code and R output, fill in the blanks with the most appropriate words or values.

<R-code>

```
summary(model)
```

<Output>

Model: "sequential_2"

Layer (type)	Output Shape	Param #
text_vectorization_2 (TextVectorization)	(None, 50)	0
embedding_2 (Embedding)	(None, 50, 64)	1,600,000
spatial_dropout1d_2 (SpatialDropout1D)	(None, 50, 64)	0
conv1d_4 (Conv1D)	(None, 50, 128)	41,088
conv1d_5 (Conv1D)	(None, 50, 128)	82,048
global_max_pooling1d_2 (GlobalMaxPooling1D)	(None, 128)	0
dropout_4 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dropout_5 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 5,194,373 (19.81 MB)
 Trainable params: 1,731,457 (6.60 MB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 3,462,916 (13.21 MB)

The number of total parameters: 5,194,373

Activation function used in the hidden layer: relu

Activation function used in the output layer: sigmoid

Input dimension of the embedding layer: 25000

Output dimension of the embedding layer: 64

Optimizer: adam

Loss function: binary_crossentropy

Number of epochs = 5

Number of samples per gradient update (batch size) = 64

2. Compute predicted probabilities, convert them to class labels using threshold **0.6**. Then, create a **confusion matrix** and compute the **accuracy** and **specificity**.

```
<R-code>
pred_prob <- model |> predict(x_test)
pred_lab <- as.integer(pred_prob > 0.6)
print(table(Predicted = pred_lab, Actual = y_test))
mean(pred_lab == y_test)
sum(y_test == 0 & pred_lab == 0) / sum(y_test == 0)
```

```
<Output>
      Actual
Predicted 0   1
      0 172  74
      1  39  84
[1] 0.6937669
[1] 0.8151659
```

Accuracy: 0.6937669
Specificity: 0.8151659

3. Print the labels, predicted probabilities, and raw text for the **two** observations predicted to be the most likely be classified as class 1.

```
<R-code>
eg_idx <- tail(order(pred_prob), 2)
for (i in eg_idx) {
  cat("\n--- Example -----Wn")
  cat("True:", y_test[i], "Wn")
  p <- as.numeric(model |> predict(x_test[i], verbose = 0))
  cat(sprintf("Pred prob (positive=1): %.3fWn", p))
  cat("Text:", x_test[i], "Wn")
}
```

```
<Output>
--- Example -----
True: 1
Pred prob (positive=1): 1.000
Text:  11월 극장에서 꼭 봐야할 유일한 영화라고 생각합니다!!!헝거게임 마지막편은 꼭 극장에서 확인
하세요!!

--- Example -----
True: 1
Pred prob (positive=1): 1.000
Text:  긴박감과 긴장감이 최고예요~ 헝거게임 꼭 보시기를! 후회 없습니다
```

4. Using the *Auto.csv* dataset, perform K-Means clustering with $k = 5$ on the **mpg** and **displacement** variables. Before clustering, standardize both variables using standard scaling. Then, using the *ggplot2* package, create a scatter plot with **mpg** on the x-axis and **displacement** on the y-axis, coloring the points by cluster (three distinct colors).

<R-code>

```
df = Auto[c("mpg", "displacement")]
km = kmeans(scale(df), centers = 5)
library(ggplot2)
df$cluster <- as.factor(km$cluster)
ggplot(df, aes(x = mpg, y = displacement, color = cluster)) +
  geom_point(size = 2) +
  theme_minimal()
```

<Output (plot)>

